

TRAVAUX PRATIQUES

Configuration SSH

Windows Server 2016 → Debian 13

PuTTY

OpenSSH

RSA 2048

Wireshark

Étudiant	Sean
Formation	BTS SIO option SISR — 1ère année
Établissement	Pôle Sup Montalembert, Courbevoie
Stage	DISI — CentraleSupélec
Date	01 avril 2026

Objectifs

- Mettre en place une connexion SSH client/serveur
- Configurer l'authentification par clé RSA
- Gérer plusieurs utilisateurs avec clés distinctes
- Analyser le trafic chiffré avec Wireshark

Sommaire

-
- | | |
|----|--|
| 01 | Environnement et prérequis
Configuration VMware, logiciels utilisés |
| 02 | Configuration réseau
LAN Segment, adressage IP statique |
| 03 | Serveur SSH
Installation et activation d'OpenSSH |
| 04 | Connexion par mot de passe
PuTTY, première connexion |
| 05 | Authentification par clé RSA
Génération, déploiement, conversion PPK |
| 06 | Multi-utilisateurs
Création de user2 avec clé distincte |
| 07 | Analyse Wireshark
Capture et preuve du chiffrement |
| 08 | Synthèse
Bonnes pratiques et conclusion |
-

1

Environnement et prérequis

Ce TP a été réalisé dans un environnement de virtualisation **VMware Workstation**. Deux machines virtuelles ont été créées et connectées via un **LAN Segment** dédié, simulant un réseau local isolé sans accès extérieur.

Composant	CLSSH (Client)	SRVSSH (Serveur)
Système	Windows Server 2016	Debian 13 (Trixie)
Rôle	Client SSH	Serveur OpenSSH
Adresse IP	10.0.0.3/24	10.0.0.2/24
RAM / CPU	2 Go / 2 vCPU	2 Go / 2 vCPU
Réseau	LAN Segment (SSH_LAN)	LAN Segment (SSH_LAN)

Logiciels utilisés



PuTTY & PuTTYgen

Client SSH pour Windows + générateur de clés
Conversion OpenSSH → format .ppk



OpenSSH Server 1:10.0p1

Serveur SSH installé sur Debian 13
Écoute sur le port 22 (TCP)



Wireshark

Analyseur de protocoles réseau
Capture et inspection des paquets SSH chiffrés

Configuration VMware

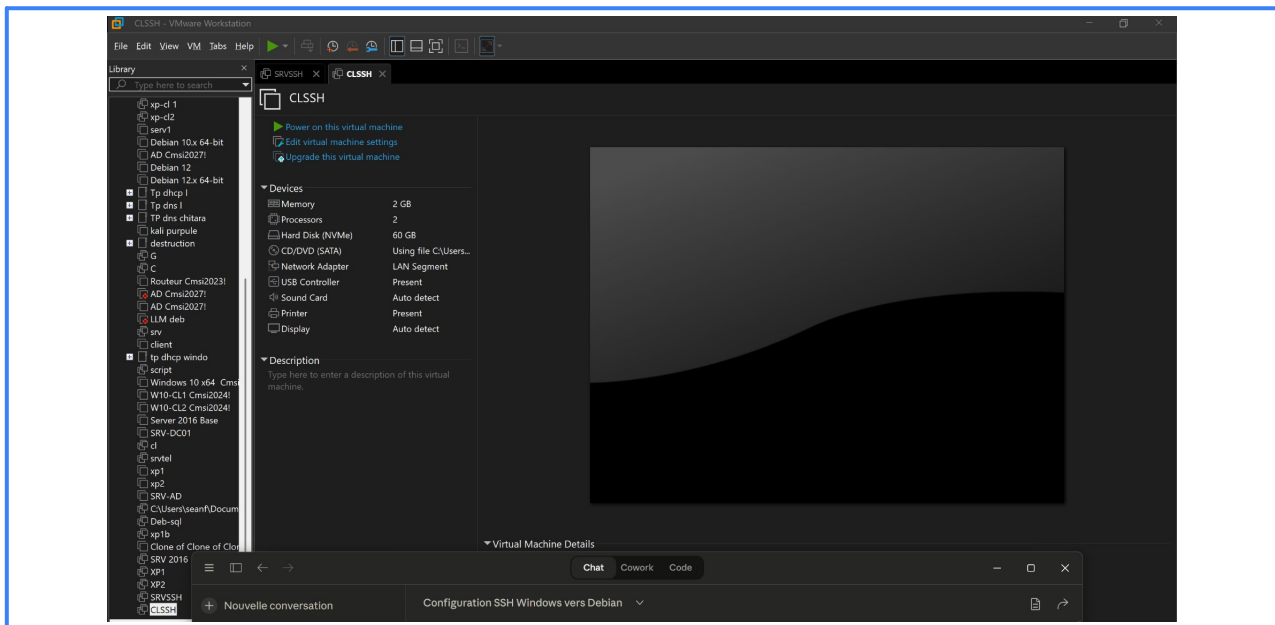
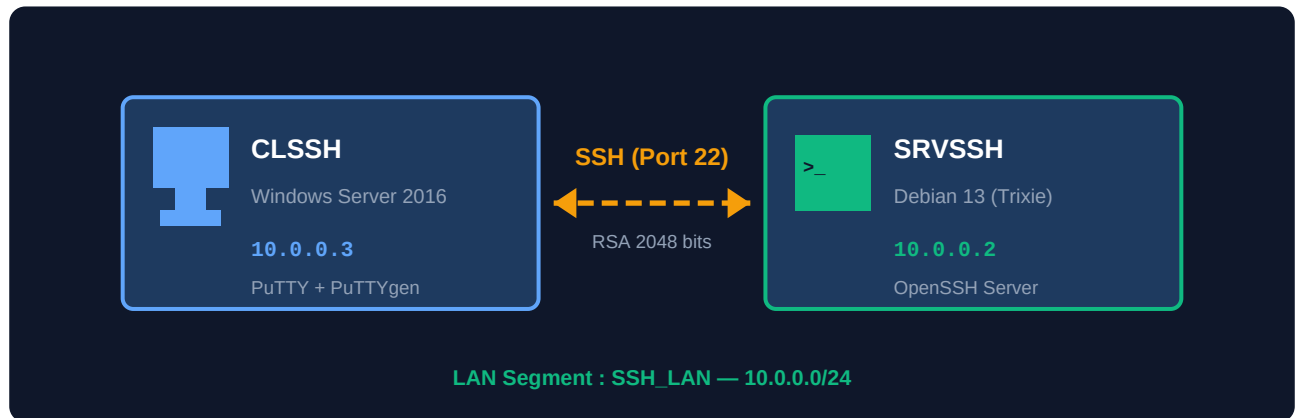


Figure 1 — Configuration de la VM CLSSH dans VMware Workstation (2 Go RAM, LAN Segment)

2 Configuration réseau

Architecture réseau du TP



Configuration IP statique — Serveur Debian

Le fichier `/etc/network/interfaces` a été modifié pour attribuer une adresse IP statique au serveur :

```
auto lo
iface lo inet loopback

auto ens33
iface ens33 inet static
    address 10.0.0.2
    netmask 255.255.255.0
```

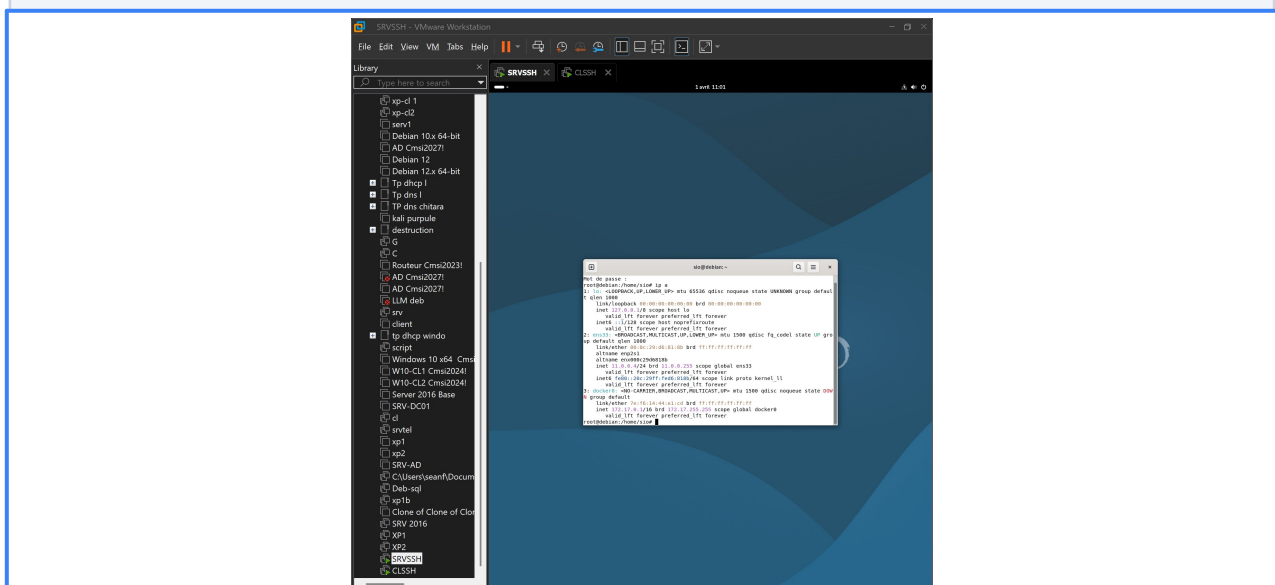


Figure 2 — Résultat de `ip a` : interface `ens33` configurée sur le serveur Debian

Configuration IP statique — Client Windows

```
netsh interface ip set address "Ethernet0" static 10.0.0.3 255.255.255.0
```

Vérification de la connectivité

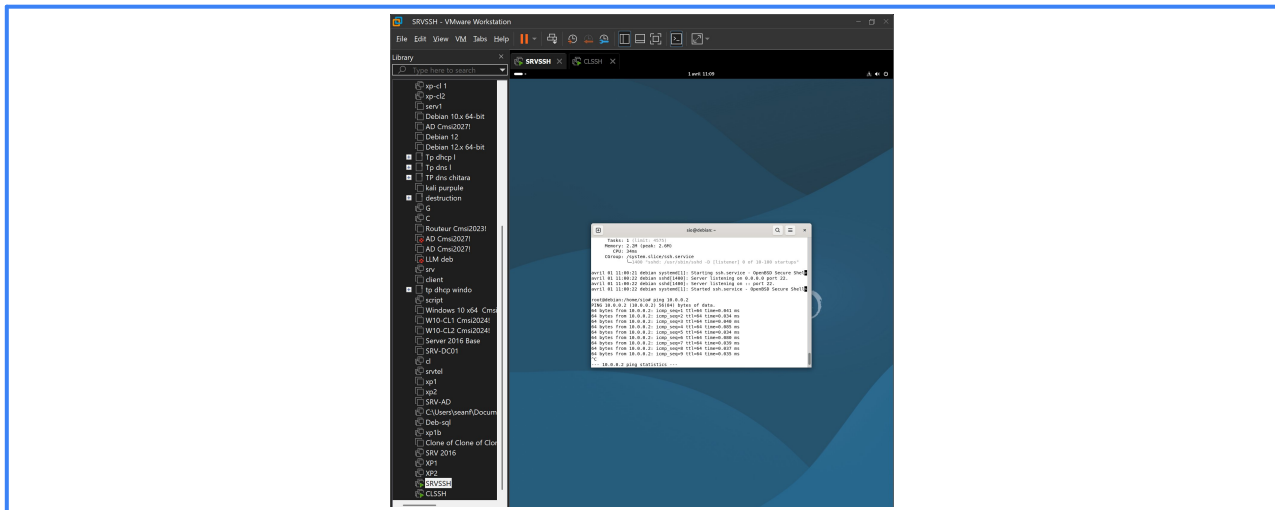


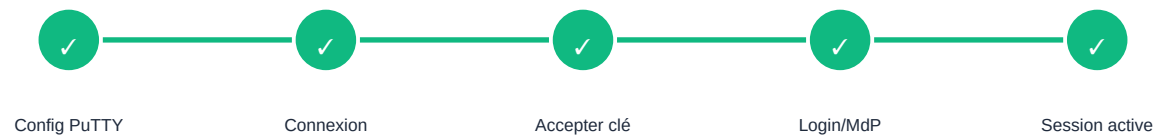
Figure 3 — Ping réussi depuis Debian vers le client Windows (10.0.0.3)

Résultat : Les deux machines communiquent correctement sur le LAN Segment SSH_LAN.

4

Connexion SSH par mot de passe

La première méthode d'authentification testée est la connexion classique par **mot de passe**. Le client PuTTY a été installé sur Windows Server 2016 et configuré pour se connecter au serveur Debian via SSH.



Configuration de PuTTY

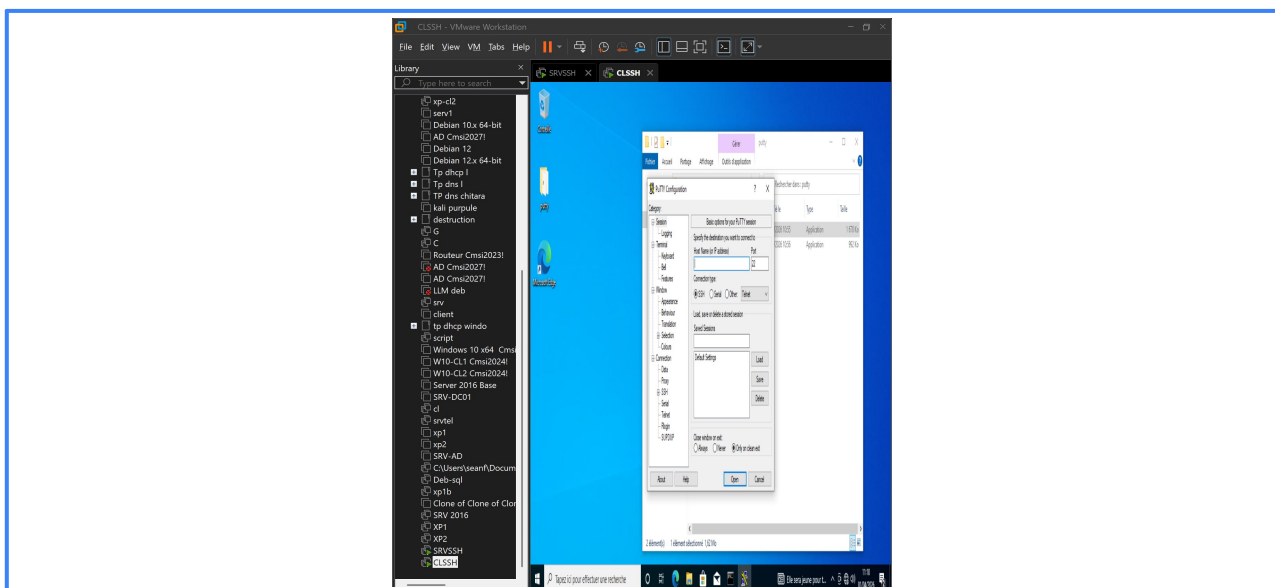


Figure 5 — PuTTY configuré : Host 10.0.0.2, Port 22, protocole SSH

Connexion réussie

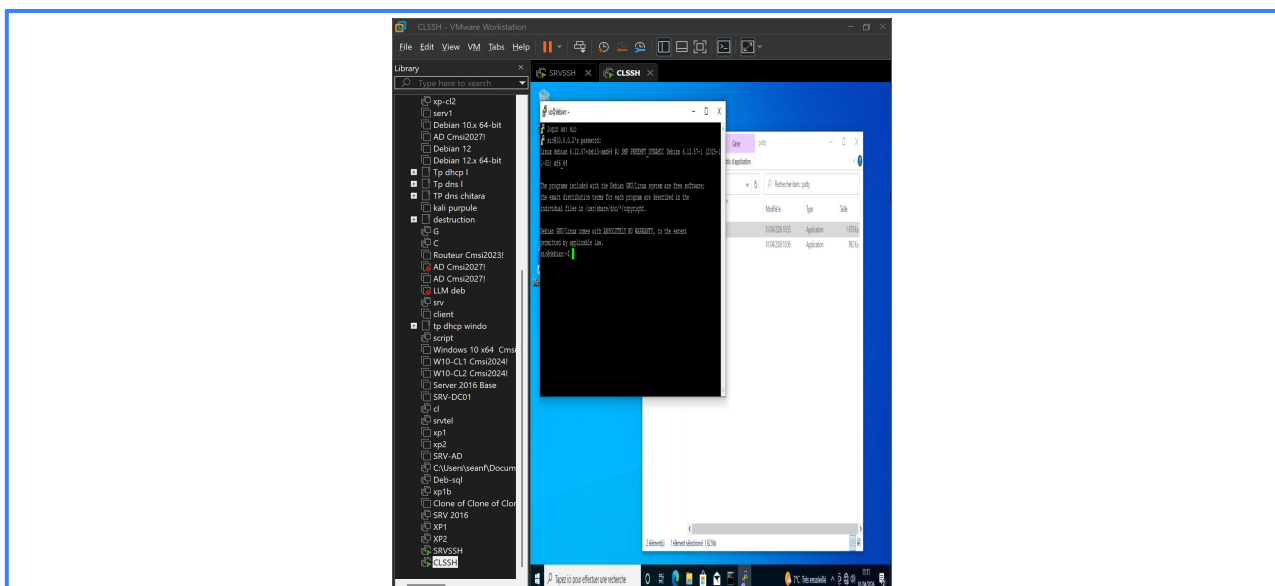


Figure 6 — Session SSH ouverte avec l'utilisateur sio (authentification par mot de passe)

Important : L'authentification par mot de passe fonctionne, mais elle est vulnérable aux attaques par force brute. La section suivante présente une méthode plus sécurisée.

5 Authentification par clé RSA

L'authentification par **clé publique/privée** est la méthode recommandée en production. Elle repose sur la **cryptographie asymétrique RSA** : seul le détenteur de la clé privée peut prouver son identité, sans jamais transmettre de secret sur le réseau.

Processus d'authentification par clé RSA



■ Clé privée = SECRET (client)

■ Clé publique = PARTAGÉE (serveur)

Génération de la paire de clés

```
ssh-keygen -t rsa -b 2048
```

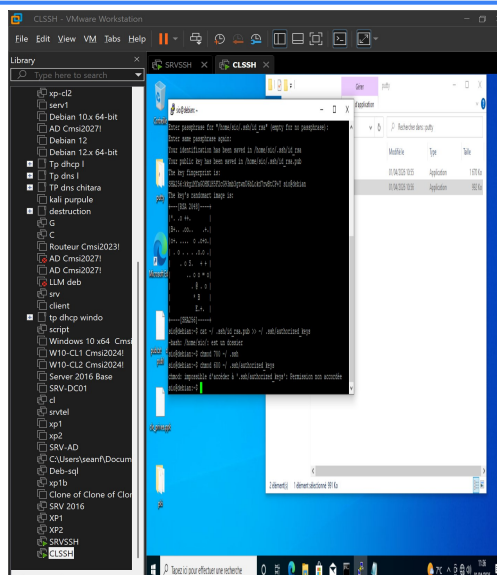


Figure 7 — Génération de la paire de clés RSA 2048 bits sur le serveur Debian

Installation de la clé publique sur le serveur

```
cp /home/sio/.ssh/id_rsa.pub /home/sio/.ssh/authorized_keys
chmod 700 /home/sio/.ssh
chmod 600 /home/sio/.ssh/authorized_keys
```

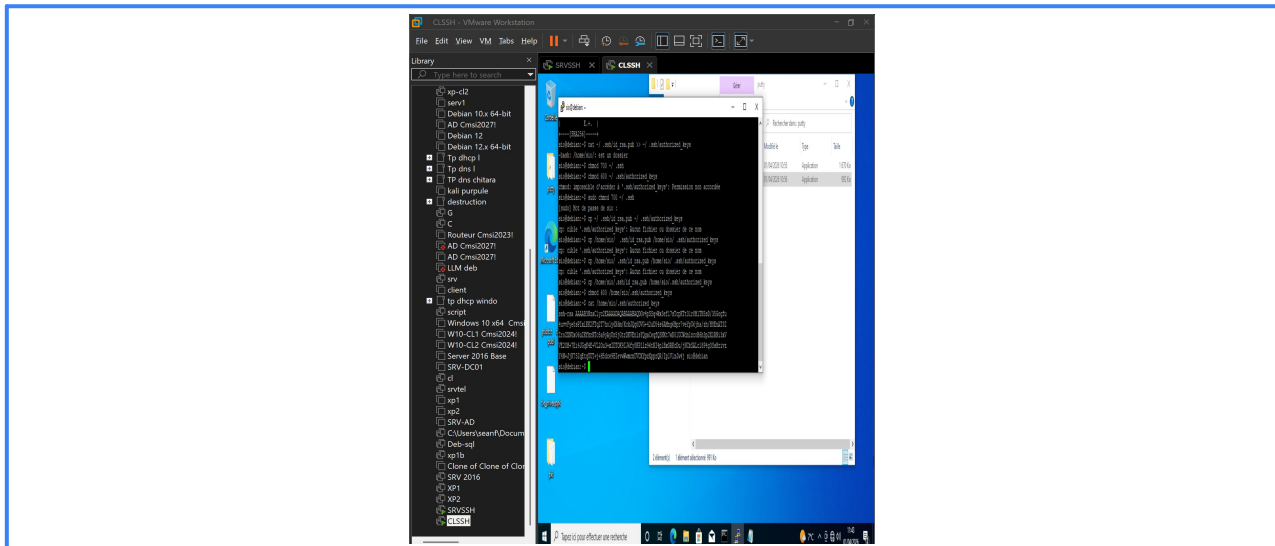


Figure 8 — Clé publique RSA installée dans le fichier `authorized_keys`

Conversion de la clé privée au format PuTTY (.ppk)

PutTY utilise son propre format de clé (**.ppk**). La clé privée OpenSSH a été transférée via un serveur HTTP temporaire (python3 -m http.server), puis convertie avec PutTYgen :

Étape	Action
1	Servir la clé via HTTP temporaire (port 8080)
2	Télécharger id_rsa depuis Edge sur Windows
3	PuTTYgen → Load → All Files → sélectionner id_rsa
4	Save private key → cle_privee.ppk

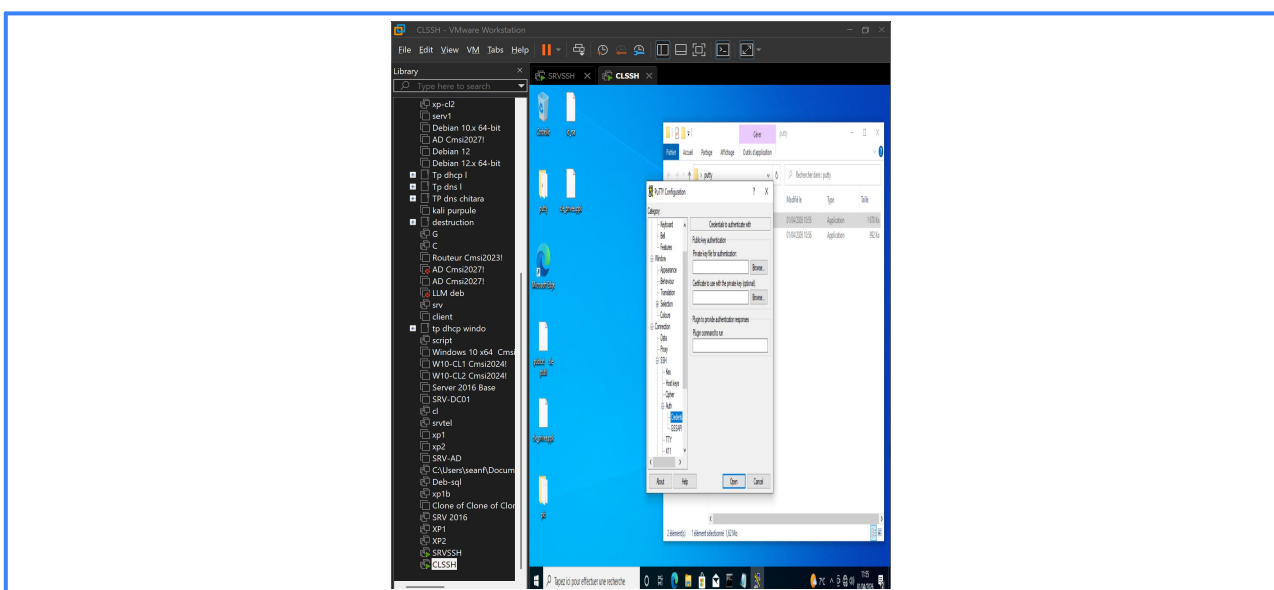


Figure 9 — PuTTY Credentials : sélection de la clé privée .ppk pour l'authentification

Connexion réussie par clé RSA

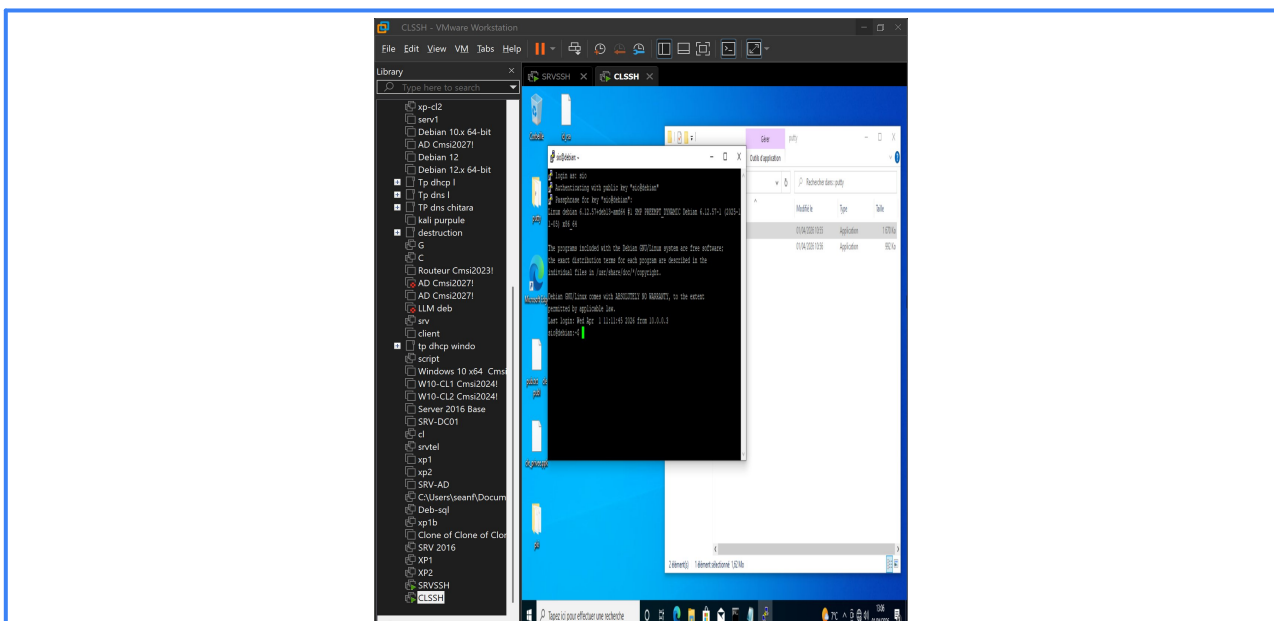


Figure 10 — « Authenticating with public key » — Connexion sio sans mot de passe

Succès : Le message « Authenticating with public key » confirme l'authentification par clé RSA. Aucun mot de passe n'est demandé.

6

Gestion multi-utilisateurs

Pour démontrer la gestion de **plusieurs utilisateurs** avec des clés distinctes, un second compte **user2** a été créé avec sa propre paire de clés RSA, indépendante de celle de l'utilisateur sio.

```
# Création de l'utilisateur
sudo adduser user2

# Génération d'une paire de clés dédiée
sudo ssh-keygen -t rsa -b 2048 -f /home/user2/.ssh/id_rsa -N ""

# Installation de la clé publique
sudo cp /home/user2/.ssh/id_rsa.pub /home/user2/.ssh/authorized_keys
sudo chown -R user2:user2 /home/user2/.ssh
sudo chmod 700 /home/user2/.ssh
sudo chmod 600 /home/user2/.ssh/authorized_keys
```

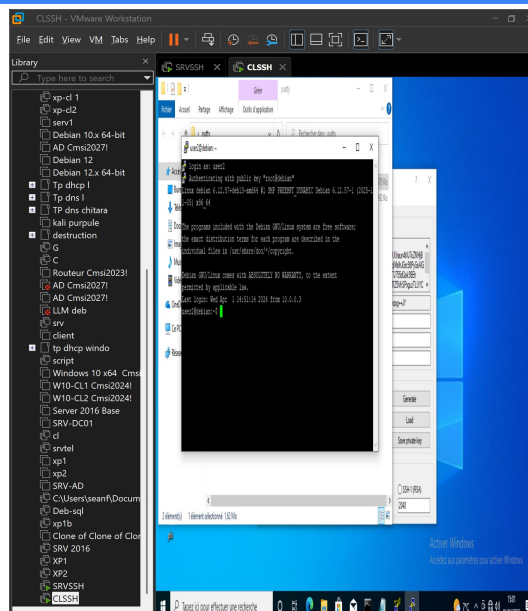


Figure 11 — Connexion SSH réussie par clé RSA pour user2 (clé distincte de sio)

Récapitulatif des accès configurés

Utilisateur	Clé privée (client)	Clé publique (serveur)	Statut
sio	cle_privée.ppk	/home/sio/.ssh/authorized_keys	✓ Actif
user2	cle_user2.ppk	/home/user2/.ssh/authorized_keys	✓ Actif

Chaque utilisateur dispose de sa propre paire de clés. Si une clé est compromise, seul l'accès de cet utilisateur est affecté. C'est le principe de **séparation des privilèges**.

7

Analyse Wireshark

Pour **prouver le chiffrement** des communications SSH, une capture réseau a été effectuée avec Wireshark sur l'interface ens33 du serveur Debian, pendant une session SSH active.

```
# Installation
apt install -y wireshark

# Lancement et capture sur ens33
wireshark &

# Filtre appliqué
tcp.port == 22
```

Capture des paquets SSH

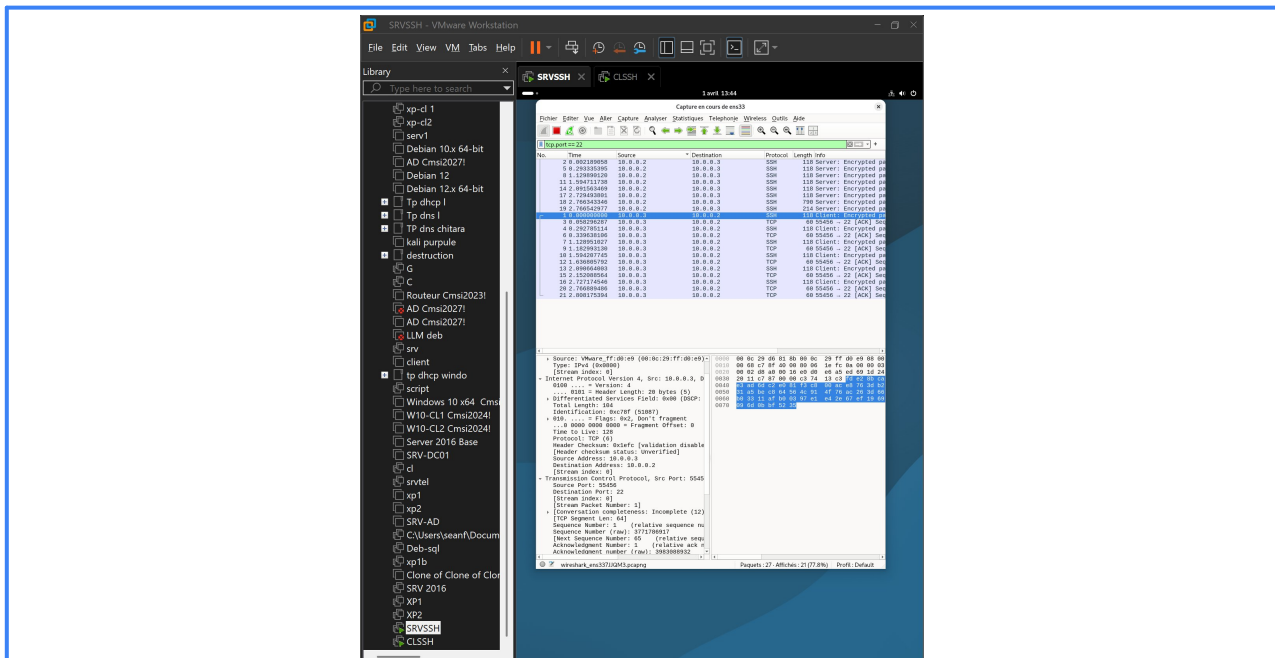


Figure 12 — Wireshark : paquets SSH entre 10.0.0.3 (client) et 10.0.0.2 (serveur)

Analyse détaillée d'un paquet

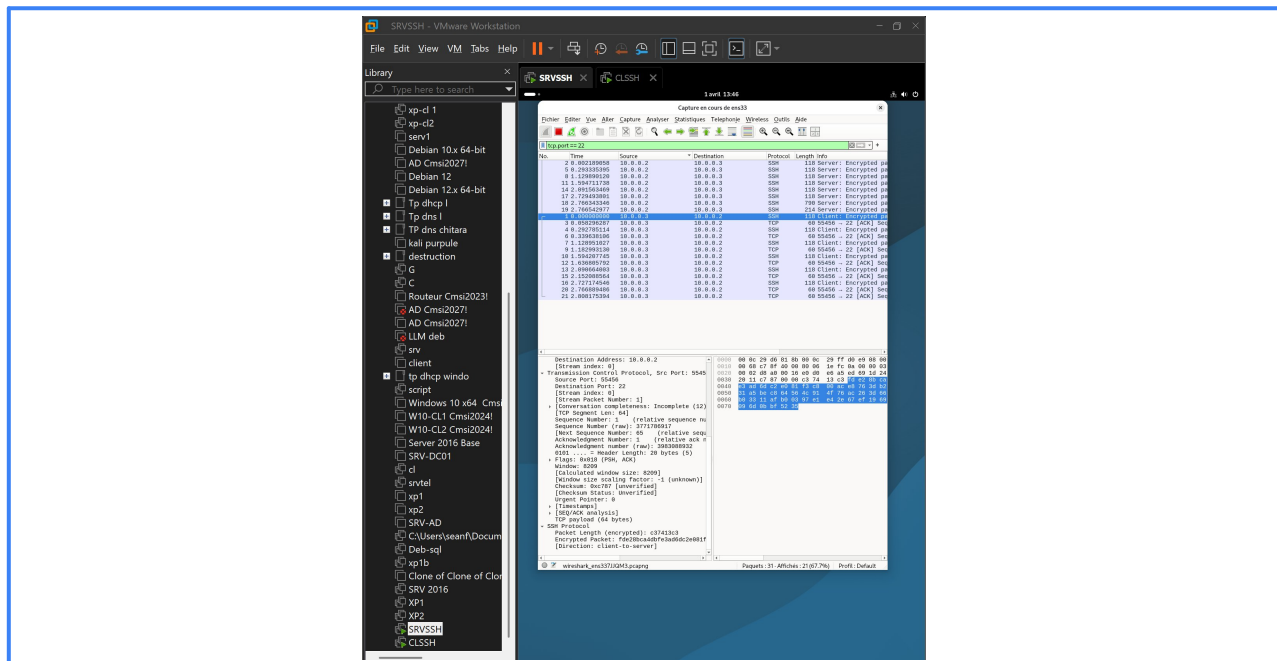


Figure 13 — Paquet SSH détaillé : « Encrypted Packet: fde28bca4dbfe3ad6dc2e081f... »

Preuve du chiffrement : Le champ « Encrypted Packet » contient une séquence hexadécimale illisible. Même en capturant tout le trafic réseau, il est **impossible** de lire les commandes tapées ou les identifiants transmis.

Comparaison visuelle : SSH vs Telnet

SSH — Sécurité : 95%

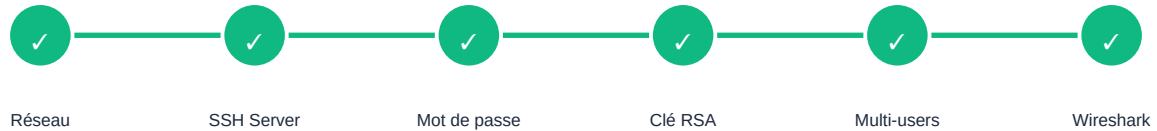
Sécurité :

5% — Aucun chiffrement

8

Synthèse et conclusion

Récapitulatif des opérations



N°	Opération	Résultat
1	Configuration du LAN Segment VMware	✓ OK
2	Attribution des adresses IP statiques (10.0.0.2/3)	✓ OK
3	Installation et activation d'OpenSSH Server	✓ OK
4	Connexion SSH par mot de passe (PuTTY)	✓ OK
5	Génération et déploiement de clés RSA 2048 bits	✓ OK
6	Authentification par clé (sio) — sans mot de passe	✓ OK
7	Création de user2 avec clé RSA distincte	✓ OK
8	Capture Wireshark — preuve du chiffrement SSH	✓ OK

Recommandations de sécurité pour la production

Mesure	Configuration	Impact
Désactiver accès root	PermitRootLogin no	Élevé
Forcer les clés uniquement	PasswordAuthentication no	Élevé
Changer le port SSH	Port 2222	Moyen
Limiter les utilisateurs	AllowUsers sio user2	Élevé
Installer fail2ban	apt install fail2ban	Élevé
Clés 4096 bits ou Ed25519	ssh-keygen -t ed25519	Moyen

Comparaison SSH vs Telnet

Critère	SSH	Telnet
Chiffrement	AES / ChaCha20	Aucun
Auth. par clé	RSA / Ed25519	Non
Port	22	23
Transfert fichiers	SCP / SFTP	Non
Interception	Impossible	Triviale

CONCLUSION

Ce TP a permis de mettre en place une **infrastructure SSH complète**, couvrant l'ensemble de la chaîne : configuration réseau, installation du serveur, authentification par mot de passe puis par clé RSA, gestion multi-utilisateurs, et vérification du chiffrement par analyse de trames.

L'authentification par **clé publique/privée RSA 2048 bits** constitue une alternative bien plus sécurisée au mot de passe : elle élimine le risque de brute force et ne transmet aucun secret sur le réseau. La capture Wireshark confirme que **l'intégralité du trafic SSH est chiffré**, rendant toute interception inexploitable.

La gestion de deux utilisateurs avec des clés distinctes illustre le principe de **séparation des privilèges**, fondamental en sécurité informatique.